Step 1 — Create Database & Collection

```
test> use bigdataDB
switched to db bigdataDB
bigdataDB> db.createCollection("sales")
... db.createCollection("customers")
{ ok: 1 }
bigdataDB>
```

Step 2 — Insert Sample Data

Insert into `sales`

```
bigdataDB> db.sales.insertMany([
... {
...   cust_id: "C101",
...   product: "Laptop",
...   category: "Electronics",
...   price: 50000,
...   qty: 2,
...   status: "completed",
...   date: ISODate("2025-01-10"),
...   items: ["mouse", "keyboard"]
... },
... {
...   cust_id: "C102",
...   product: "Mobile",
...   category: "Electronics",
...   price: 20000,
...   qty: 3,
...   status: "completed",
...   date: ISODate("2025-02-15"),
...   items: ["earphones", "charger"]
... },
... {
...   cust_id: "C101",
...   product: "Chair",
...   category: "Furniture",
...   price: 5000,
...   qty: 4,
...   status: "pending",
...   date: ISODate("2025-03-05"),
...   items: ["cushion"]
... },
... {
...   cust_id: "C103",
...   product: "Table",
...   category: "Furniture",
...   price: 8000,
...   qty: 1,
...   status: "completed",
...   date: ISODate("2025-01-25"),
...   items: ["cover"]
... }
... ])
...
{
  acknowledged: true,
  insertedIds: {
```

Insert into `customers`

```
bigdataDB> db.customers.insertMany([
... { cust_id: "C101", name: "Ravi", city: "Chennai" },
... { cust_id: "C102", name: "Anu", city: "Bangalore" },
... { cust_id: "C103", name: "John", city: "Mumbai" }
... ])
...
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('697f616f58d70522341e2625'),
    '1': ObjectId('697f616f58d70522341e2626'),
    '2': ObjectId('697f616f58d70522341e2627')
  }
}
```

What are Aggregate Operations in MongoDB?

Aggregate operations **process data records** and **return computed results**.

They are used for:

- Summarization (sum, avg, count)
- Grouping data
- Filtering
- Transforming documents
- Joining collections
- Time-series and analytical queries

MongoDB uses the **Aggregation Pipeline Framework** for this.

Aggregation Pipeline Concept

Documents pass through multiple stages.

Output of one stage becomes input to next.

Think of it like a **data processing assembly line**:

Collection → Stage1 → Stage2 → Stage3 → Output

Each stage:

- Takes input documents
- Processes them
- Passes results to next stage

```
db.collection.aggregate([

  { <stage1> },

  { <stage2> },

  ...

])
```

```
bigdataDB> db.sales.find().pretty()
... db.customers.find().pretty()
...
[
  {
    _id: ObjectId('697f616f58d70522341e2625'),
    cust_id: 'C101',
    name: 'Ravi',
    city: 'Chennai'
  },
  {
    _id: ObjectId('697f616f58d70522341e2626'),
    cust_id: 'C102',
    name: 'Anu',
    city: 'Bangalore'
  },
  {
    _id: ObjectId('697f616f58d70522341e2627'),
    cust_id: 'C103',
    name: 'John',
    city: 'Mumbai'
  }
]
bigdataDB> db.sales.find().pretty()
[
```

1. Basic Aggregation — `$group` + `$sum`

Part	Meaning
`$group`	Perform aggregation
`_id: null`	Group all documents together
`$sum: "$qty"`	Add all `qty` values
`totalQty`	Name of output field

Force output display in mongosh----------.toArray()

1) Basic Aggregation — Total Quantity

```
bigdataDB> db.sales.aggregate([
...   {
...     $group: {
...       _id: null,
...       totalQty: { $sum: "$qty" }
...     }
...   }
... ]).toArray()
...
[ { _id: null, totalQty: 10 } ]
```

Store result and print only the number

```
bigdataDB> var result = db.sales.aggregate([
...   {
...     $group: {
...       _id: null,
...       totalQty: { $sum: "$qty" }
...     }
...   }
... ]).toArray()
...
... print(result[0].totalQty)
...
10
```

2) Total Quantity per Customer

cust_id	product	qty
C101	Laptop	2
C102	Mobile	3
C101	Chair	4
C103	Table	1

Customer C101 bought how many items?
Customer C102 bought how many items?

Customer C103 bought how many items?

- `$group` → grouping happens
- `_id` → grouping field
- `$sum` → calculation inside each group

```
bigdataDB> db.sales.aggregate([
...   {
...     $group: {
...       _id: "$cust_id",
...       totalQty: { $sum: "$qty" }
...     }
...   }
... ]).toArray()
...
[
  { _id: 'C103', totalQty: 1 },
  { _id: 'C101', totalQty: 6 },
  { _id: 'C102', totalQty: 3 }
]
bigdataDB>
```

3) Total Sales Amount (price × qty)
   Step 1: Create amount for every document
   Step 2: Add all amounts

- `$project` → create new computed field
- `$multiply` → arithmetic operator
- `$group` + `$sum` → aggregate result

```
bigdataDB> db.sales.aggregate([
...   {
...     $project: {
...       amount: { $multiply: ["$price", "$qty"] }
...     }
...   },
...   {
...     $group: {
...       _id: null,
...       totalSales: { $sum: "$amount" }
...     }
...   }
... ]).toArray()
...
[ { _id: null, totalSales: 188000 } ]
bigdataDB>
```

To cleaner output

```
bigdataDB> db.sales.aggregate([
...   {
...     $project: {
...       amount: { $multiply: ["$price", "$qty"] }
...     }
...   },
...   {
...     $group: {
...       _id: null,
...       totalSales: { $sum: "$amount" }
...     }
...   },
...   {
...     $project: { _id: 0, totalSales: 1 }
...   }
... ]).toArray()
...
[ { totalSales: 188000 } ]
bigdataDB>
```

4) Average, Min, Max Price per Category

- Grouping by a field
- Using **multiple aggregate operators** together
- One `$group` can perform many calculations at once

```
bigdataDB> db.sales.aggregate([
...   {
...     $group: {
...       _id: "$category",
...       avgPrice: { $avg: "$price" },
...       minPrice: { $min: "$price" },
...       maxPrice: { $max: "$price" }
...     }
...   }
... ]).toArray()
...
[
  {
    _id: 'Electronics',
    avgPrice: 35000,
    minPrice: 20000,
    maxPrice: 50000
  },
  { _id: 'Furniture', avgPrice: 6500, minPrice: 5000, maxPrice: 8000 }
]
bigdataDB>
```

5) Extract Year and Month from Date

Word	Purpose
`aggregate`	Start aggregation
`$project`	Reshape document
`1`	Include field
`$year`, `$month`	Extract from date
`"$date"`	Refer to date field
`toArray()`	Show output

```
bigdataDB> db.sales.aggregate([
...   {
...     $project: {
...       product: 1,
...       year: { $year: "$date" },
...       month: { $month: "$date" }
...     }
...   }
... ]).toArray()
...
[
  {
    _id: ObjectId('697f612658d70522341e2621'),
    product: 'Laptop',
    year: 2025,
    month: 1
  },
  {
```

`$project` is a **temporary view** created during aggregation.

6) Work with Array — `$unwind`

Sample Data (from `sales`)

product	items
Laptop	["mouse", "keyboard"]
Mobile	["earphones", "charger"]
Chair	["cushion"]
Table	["cover"]

We want to know:

"How many times each item was sold?"

But problem is: `items` is an **array**.

MongoDB cannot group arrays directly.

Step 1: Break arrays into rows
Step 2: Group those rows
Step 3: Count them

Operator	Role
`$unwind`	Break array into documents
`$group`	Group items
`$sum: 1`	Count occurrences

```
bigdataDB> db.sales.aggregate([
...   { $unwind: "$items" },
...   {
...     $group: {
...       _id: "$items",
...       count: { $sum: 1 }
...     }
...   }
... ]).toArray()
...
[
  { _id: 'charger', count: 1 },
  { _id: 'keyboard', count: 1 },
  { _id: 'mouse', count: 1 },
  { _id: 'earphones', count: 1 },
  { _id: 'cushion', count: 1 },
  { _id: 'cover', count: 1 }
]
bigdataDB>
```

7) Sort and Limit (Top Costly Products)

```
bigdataDB> db.sales.aggregate([
...   { $sort: { price: -1 } },
...   { $limit: 2 }
... ]).toArray()
...
[
  {
    _id: ObjectId('697f612658d70522341e2621'),
    cust_id: 'C101',
    product: 'Laptop',
    category: 'Electronics',
    price: 50000,
    qty: 2,
    status: 'completed',
    date: ISODate('2025-01-10T00:00:00.000Z'),
    items: [ 'mouse', 'keyboard' ]
  },
  {
    _id: ObjectId('697f612658d70522341e2622'),
    cust_id: 'C102',
    product: 'Mobile',
    category: 'Electronics',
    price: 20000,
    qty: 3,
    status: 'completed',
    date: ISODate('2025-02-15T00:00:00.000Z'),
    items: [ 'earphones', 'charger' ]
```

Operator	Role
`$sort`	Order documents
`$limit`	Restrict number of results

Stage	Purpose
`$group`	Create buckets and calculate
`$project`	Show fields and create new ones
`$unwind`	Break arrays
`$sort`	Order data
`$limit`	Top N results
`$year`, `$month`	Date analysis

- `$group` → “Put similar data together”
- `$project` → “Temporary view”
- `$unwind` → “Break array into rows”
- Pipeline → “Assembly line processing”